Chapter 23: Product Metrics (Part 2)

Software Engineering

Program: BTech Computer Science and Engineering

Duration: 1 Hour

Advanced Product Metrics

From Foundation to Deep Dive

PREVIOUSLY COVERED

- The **foundation** of product metrics (Measures, Metrics, and the GQM paradigm).
- **Function points** for requirements.
- Basic **Object-Oriented (OO)** metrics.

TODAY'S FOCUS

- **Diving deeper** into specific, advanced metric suites.
- Moving from **design** to implementation and maintenance.

THE GOAL

- **Predict software quality.**
- Guide **continuous improvement.**
- Make strictly **data-driven** engineering decisions.

Lecture Objectives

Mastering Advanced Metric Suites

- Apply the CK (Chidamber & Kemerer) Metrics Suite, MOOD Metrics Suite, and Lorenz & Kidd metrics for advanced Object-Oriented design.
- Describe component-level design metrics and operation-oriented metrics.
- Explain specific design metrics tailored for WebApps and User Interfaces (UI).
- Use source code metrics (such as Halstead metrics and cyclomatic complexity) to evaluate implementation quality.
- Understand the metrics used to track and optimize software testing and maintenance activities.

The CK Metrics Suite

A Validated Framework for OO Design

The Industry Standard: The Chidamber and Kemerer (CK) suite remains the most widely validated and utilized metrics suite for Object-Oriented design.

The Core Application: We don’t just calculate these numbers for academic purposes. We use them to actively predict:

- Maintainability: How difficult and expensive will it be to update this code later?
- Testability: How much effort will QA need to exert to ensure it works?

Complexity & Inheritance

WMC, DIT, and NOC

Metric	Definition	Real-World Interpretation
WMC (Weighted Methods per Class)	The sum of the complexities of all methods within a single class.	High WMC → Indicates a “heavy” class requiring significantly more testing effort and higher long-term maintenance costs.
DIT (Depth of Inheritance Tree)	The maximum path length from the current class up to the root parent class.	High DIT → Means more inherited behavior. It promotes reuse, but makes the class much harder to understand and test.
NOC (Number of Children)	The total count of immediate subclasses branching off a parent class.	High **NOC** → Shows high potential for reuse, but means any change to the parent requires massive regression testing of all children.

Class Interactions and Focus

Coupling, Response, and Cohesion
CBO, RFC, and LCOM

Metric	Definition	Real-World Interpretation
CBO (Coupling Between Objects)	The number of other classes to which this specific class is coupled.	High CBO → The class is brittle, highly sensitive to outside changes, and suffers from low reusability.
RFC (Response for a Class)	The total number of methods that can be invoked in response to a message.	High RFC → Creates a massive chain reaction, leading to complex testing and a higher likelihood of defects.
LCOM (Lack of Cohesion of Methods)	The number of method pairs inside the class that do not share attributes.	High LCOM → The class is acting as a "God class" doing multiple unrelated things. It should immediately be split.

The MOOD Metrics Suite

System-Level Object-Oriented Design

What is it?

MOOD stands for Metrics for Object-Oriented Design, originally proposed by Fernando Brito e Abreu.

The Core Focus

Unlike the CK suite which often looks at individual classes, MOOD provides a macro, system-level view of your design.

The Four Pillars

It specifically measures the application of four core OOP principles across the entire system:

- **Encapsulation** (Information Hiding)
- **Inheritance** (Reuse)
- **Polymorphism** (Flexibility)
- **Coupling** (Interdependence)

Encapsulation Metrics

Method and Attribute Hiding Factors

Encapsulation is measured by looking at how much of the system is kept private versus exposed to the public.

Metric	Definition	Interpretation
Method Hiding Factor (MHF)	The proportion of methods in the system that are explicitly declared as private (not inherited or overridden).	High MHF → Indicates strong encapsulation and secure internal workings, but can lead to a rigid system with less flexibility.
Attribute Hiding Factor (AHF)	The proportion of attributes (variables) in the system that are explicitly declared as private.	High AHF → Excellent information hiding. Data is highly protected from unauthorized outside interference.

Inheritance Metrics

Method and Attribute Inheritance Factors

Inheritance metrics reveal how heavily the system relies on reusing parent code.

Metric	Definition	Interpretation
Method Inheritance Factor (MIF)	The ratio of inherited methods to the total number of methods available in the system.	High MIF → High code reuse. However, excessive inheritance creates a highly complex, deep hierarchy that is difficult to trace.
Attribute Inheritance Factor (AIF)	The ratio of inherited attributes to the total number of attributes available.	High AIF → Similarly indicates strong reuse of data structures, but actively increases the coupling between parent and child classes.

Polymorphism and Coupling Metrics

System flexibility and interdependence.

This slide evaluates how polymorphism is implemented and the level of coupling within the system.

Metric	Definition	Interpretation
Polymorphism Factor (PF)	The proportion of methods across the system that are actively overridden by subclasses.	High PF → The system is highly dynamic and flexible. However, this flexibility requires significantly more complex testing to cover all execution paths.
Coupling Factor (CF)	The ratio of actual class couplings to the maximum possible number of couplings in the system.	High CF → A massive red flag. The system has high interdependence, meaning changes will cause ripple effects, resulting in very low maintainability.

The Lorenz and Kidd Metrics Suite

Practical Project-Level Monitoring

The Focus: While other suites focus heavily on abstract architectural theory, Lorenz and Kidd focus on practical, project-level metrics.

The Goal: To provide project managers and lead developers with quantifiable targets for planning, estimating effort, and monitoring daily progress.

The Application: These metrics act as “rules of thumb” to quickly identify classes that are growing out of control and need immediate refactoring.

Class Size Metrics

Keeping Objects Manageable

Lorenz and Kidd establish strict thresholds to prevent “God classes” from forming.

Metric	Definition	Target Threshold
Number of Operations	The average number of operations (methods) contained within a single class.	**Target: < 20** Classes with 20+ operations are doing too much and should be split.
Number of Attributes	The average number of attributes (variables/state) per class.	**Target: < 6** (for analysis classes) Too many attributes implies the class lacks focused cohesion.

Inheritance & Reuse Metrics

Measuring True Code Leverage

Inheritance Metrics:

- **Number of Subclasses per Class:** Averages the immediate children. Helps identify over-used parent classes.
- **Depth of Inheritance Hierarchy:** Tracks the overall depth to ensure the system doesn't become impossible to trace.

Reuse Metrics (Tracking Effort):

- **Number of New Operations:** The percentage of operations that are written from scratch (not inherited).
- **Number of Overridden Operations:** The percentage of inherited operations that are overridden.

Component-Level Design Metrics

Evaluating Internal Quality

Assessing the internal structural strength, interdependency, and complexity of individual software components—such as modules, functions, or classes.

Identifying potential maintenance bottlenecks, testing challenges, or design flaws early in the implementation phase.

Cohesion Metrics

Measuring Internal Strength

Cohesion assesses how closely related and focused the responsibilities are within a single component.

High cohesion means a component does one thing very well.

Key Metrics:

- **Cohesion of Module (CM):** A general metric assessing how well a module's internal elements work together to accomplish a single goal.
- **LCOM (Lack of Cohesion of Methods):** (Already covered) Measures whether a class's methods share the same internal attributes.

Chapter 23

Coupling Metrics

Measuring Interdependency

Definition: Coupling measures the degree of interdependency between components. Our golden rule is to seek Low Coupling.

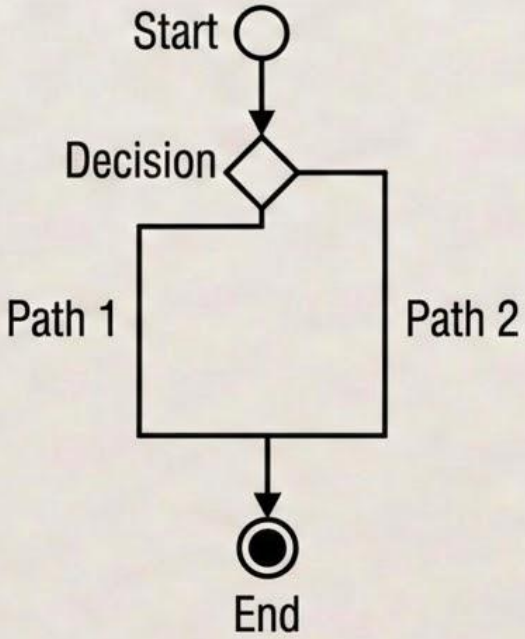
- **Coupling Between Modules (CBM):** A numerical count of all distinct connections and dependencies leading into and out of a module.
- **Data Coupling (LOW RISK):** Modules communicate solely via simple, elementary data parameters (e.g., passing integers or strings).
- **Control Coupling (HIGH RISK):** Modules communicate by passing 'flags' that affect the internal logical flow of another component.

Chapter 23

Complexity Metrics

Measuring Decision Paths

Definition: Complexity measures how difficult a component is to understand, maintain, or test mathematically.



- **Cyclomatic Complexity (V(G)):** Measures the number of independent, linear paths through a program's control flow control flow structure.

Chapter 23

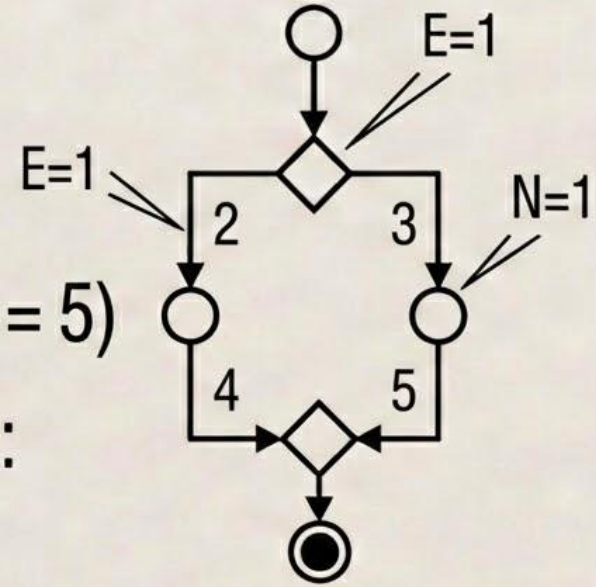
V(G) Control Flow Tool

Edges, Nodes, and Paths

$$V(G) = E - N + 2$$

E = Count of Edges (control flow paths)
N = Count of Nodes (logic blocks) (e.g., N = 5)

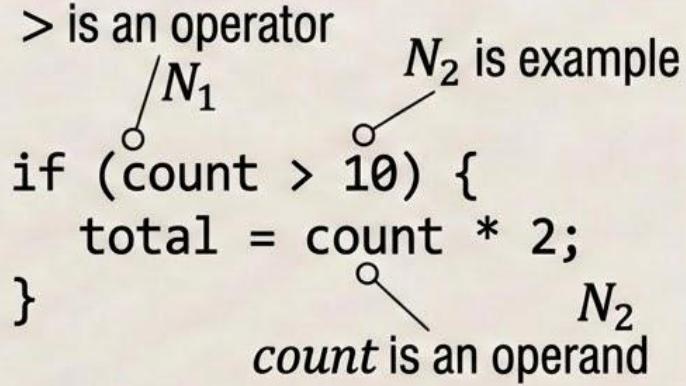
- **McCabe Thresholds** (Risk Assessment): $V(G) \leq 10$ is usually ideal.



Halstead Metrics

Source Code Science

- **Introduction & Definition:** Proposed by Maurice Halstead in 1977, these metrics are derived strictly from analyzing the source code tokens (operators and operands) rather than structural logic.
- **The Basis (Token Counts):**
  - n_1: Count of Distinct Operators
  - n_2: Count of Distinct Operands
  - N_1: Total Operator Occurrences
  - N_2: Total Operand Occurrences



Distinct Operators (n_1):
[if, >, {, total, =, count, *, 2, }, ;]
Distinct Operands (n_2):
[count, 10, total, 2]

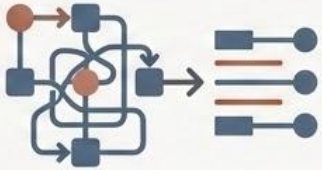
THE MICRO VIEW

Operation-Oriented Metrics

Measuring the Complexity of Individual Methods

THE FOCUS

Zooming in from the class level to evaluate the internal complexity and design quality of individual operations (methods or functions).



WHY IT MATTERS

Even in a perfectly structured Object-Oriented system, overly complex individual methods will inevitably create testing bottlenecks and harbor hidden defects.

CORE METRIC

$$\text{Average Operation Complexity} = \frac{\text{WMC}}{\text{Number of operations}}$$

(Provides a baseline expectation for how heavy a typical method is within a specific class).

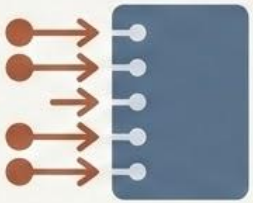
THE MICRO VIEW

Operation Arguments

Evaluating the Interface

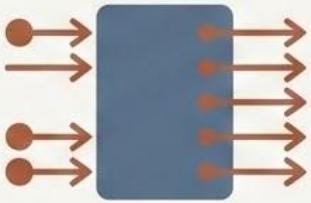
THE FOCUS

The signature of an operation often dictates how difficult it will be to integrate and test.



INPUT PARAMETERS

- **THE RULE:** More parameters → a more complex interface.
- **THE RISK:** High input counts require exponentially more test cases to cover all possible combinations.



RETURN PARAMETERS

- **THE RULE:** More returns → higher internal and external complexity.
- **THE RISK:** Functions should ideally have a single, predictable return state. Multiple returns often indicate the method is doing too much.

THE MICRO VIEW

Operation Size

Lines of Code and Message Passing

LOC PER OPERATION

- A traditional measure of sheer volume.
- **CAVEAT:** Benchmarking useful within a codebase, not ideal across programming languages.

NUMBER OF MESSAGES SENT

- Counts other operations this operation calls.
- **THE RISK:** Tight coupling to other methods; “traffic cop” role.

THE MICRO VIEW

Advanced Product Metrics

User Interface (UI) Design Metrics

Previously Covered: Advanced Object-Oriented design and component-level metrics.

Today's Shift: Moving from internal code structure to the external user experience.

THE GOAL

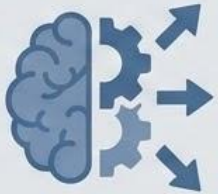
Measuring how effectively and efficiently users can interact with our software.

The Goal of UI Metrics

From Subjective to Objective Usability

PRIMARY OBJECTIVE

To quantitatively measure and track the usability and overall functional quality of a user interface.



DATA-DRIVEN DECISIONS

Moving beyond opinions like ‘I don’t like this screen’ to specific data points like ‘This form has too many fields.’

METRICS ARE INDICATORS

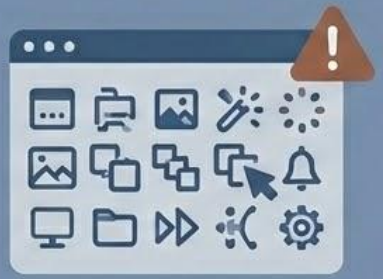
A high usability metric score indicates potential issues:

- User confusion or fatigue.
- Increased cognitive load.
- Slower task completion times.
- A higher rate of fatal user errors.

UI Layout Metrics

Number of Icons and Menu Items per Screen

NUMBER OF ICONS



COGNITIVE LOAD

NUMBER OF MENU ITEMS

NUMBER OF ICONS | Visual noise &

- clutter
- overwhelming users and decreasing findability of functions.

NUMBER OF MENU ITEMS |

- Significantly increases cognitive load
- Decision paralysis and users get lost in navigation.

Navigation Metrics

User Effort & Navigation Paths | Clicks, Screens, and Destination Depth

NUMBER OF SCREENS PER TASK

PRINCIPLE: The more screens a user must click through to complete a standard task (e.g., checkout), the more physical and mental effort is required.

GOAL:

- Simplify paths to minimize friction.

NAVIGATION DEPTH

PRINCIPLE: How many total clicks/screens are required for a user to reach a specific destination or function.

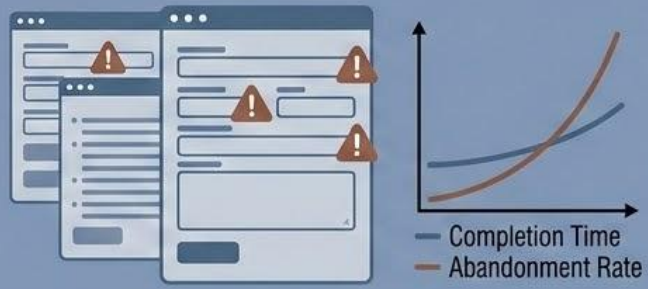
- **GOAL:** Keep essential, frequent functions within 1-3 clicks.
- Excessive depth indicates critical functions are “buried”.

Interaction Metrics and Error Rate

Quantifying User Errors and Fatigue

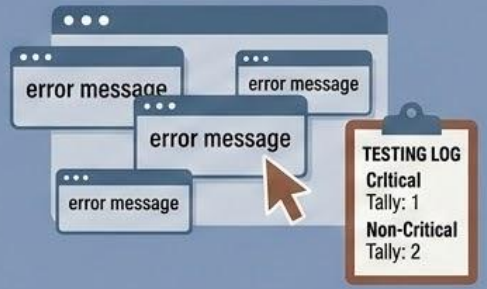
Interaction metrics move beyond static layout to evaluate how **efficiently** a user can perform **input-intensive** operations.

NUMBER OF INPUT FIELDS PER FORM



USER FRICTION

ERROR RATE (KEY TESTING INDICATOR)



- **The Direct Correlation:** Completion time and form abandonment rates directly correlate to the total number of input fields.
- **The Insight:** Every unnecessary field is an opportunity for a user to experience fatigue and error.

- **The Definition:** The frequency of critical and non-critical user errors recorded during structured usability testing sessions.
- **The Value:** Provides direct, measurable evidence of design flaws that are causing functional user failure.

Design Metrics for WebApps

Measuring the Dynamic Web

THE WEBAPP DIFFERENCE:

Web applications cannot be measured the same way as traditional desktop software.

UNIQUE CHARACTERISTICS:

CONTENT

CONTENT

The sheer volume of media, text, and data.

NAVIGATION

NAVIGATION

How users move through that vast vast content network.

PRESENTATION

PRESENTATION

How the application dynamically renders across different devices and browsers.

Content Metrics

Managing the Digital Footprint

CONTENT QUANTITY & SIZE

NUMBER OF OBJECTS

The total count of distinct HTML pages, images, videos, and scripts managed by the system.

- Indicates storage needs and CMS complexity.

CONTENT OBJECT SIZE

The average size of these objects (measured in bytes, words, or pages).

- Helps predict bandwidth requirements and server load.

LINK INTEGRITY

- **The Definition:** The percentage of internal and external links that are valid and active.
- **The Value:** A critical measure of site maintenance; broken links instantly degrade user trust and SEO rankings.

Navigation Metrics

Evaluating the Site Architecture

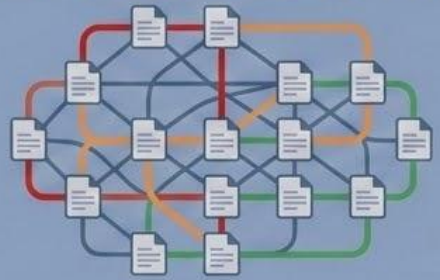
NAVIGATION DEPTH

- Average number of clicks required to reach specific content from the homepage.
- Target: Keep essential content shallow (1-3 clicks).

PAGE CONNECTIVITY

- Number of internal links pointing to and from a specific page.
- Indicates the ‘hub’ status of a page. Highly connected pages are critical to user flow.

NUMBER OF NAVIGATION PATHS



- The total number of possible logical paths a user can take through the site architecture.
- High path counts mean the site is highly interconnected, but can also complicate usability testing.

Slide 4:

Presentation and Performance Metrics

Rendering and Reliability

PRESENTATION METRICS

Number of Page Templates

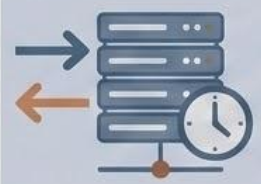
The count of distinct visual layouts the site uses.

- Fewer templates = easier maintenance & visual consistency.

PERFORMANCE METRICS

Page Load Time

How long to render a page under varying network speeds (5G vs. broadband).



Response Time

The strict server-side measurement from request received to first byte sent.

PERFORMANCE METRICS

Throughput

The volume of transactions processed per second (TPS).

- Crucial measure of site responsiveness and backend reliability.

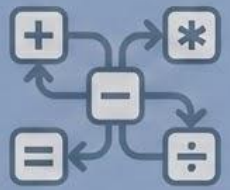
Metrics for Source Code

The Science of Software Tokens

The Paradigm Shift: Evaluating source code not by logical structure, but by a literal language.

Halstead's Software Science (1977): Proposed by Maurice Halstead, this method evaluates code by counting its fundamental linguistic tokens.

OPERATORS (VERBS)



- The "verbs" of the code.
- Examples: +, =, `if`, `while`, `return`

OPERANDS (NOUNS)

- The "nouns" of the code.
- Examples: variables, constants, strings like "Hello"

The Basic Halstead Measures

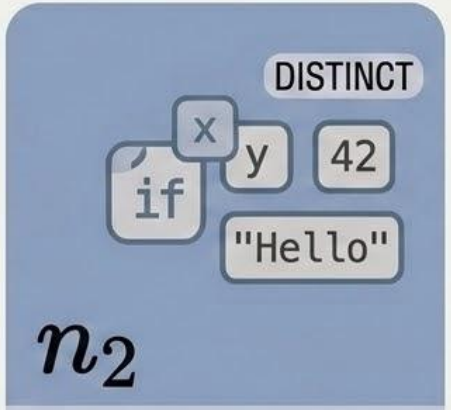
Distinct vs. Total Occurrences

Everything in Halstead's suite is derived from four basic, objective counts:

n_1

Number of distinct operators

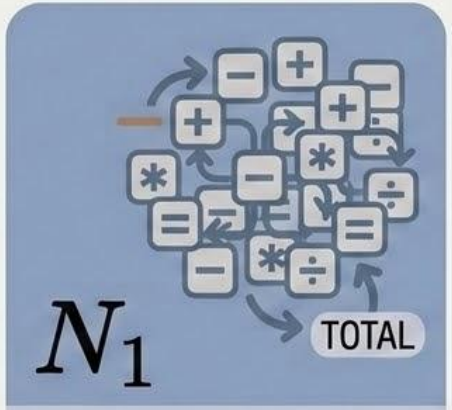
- How many unique verbs exist in the file?



n_2

Number of distinct operands

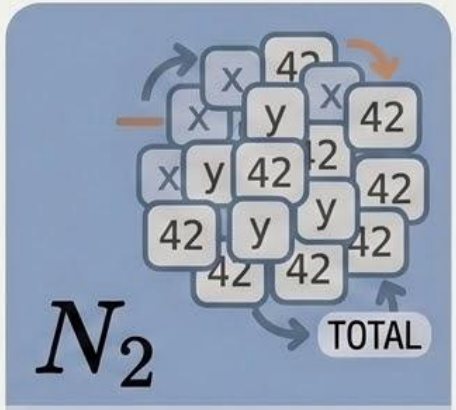
- How many unique variables/constants are used?



N_1

Total occurrences of all operators

- How many total times did we use verbs?



N_2

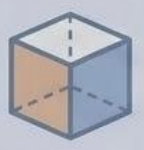
Total occurrences of all operands

- How many total times did we use nouns?

Metrics for Source Code

Derived Size Metrics

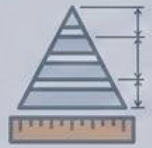
Vocabulary, Length, and Volume

Metric	Formula	What it Indicates
 Program Vocabulary (n)	$n = n_1 + n_2$	The total dictionary of unique words needed to write this module.
 Program Length (N)	$N = N_1 + N_2$	The absolute physical length of the program in total tokens.
 Program Volume (V) 	$V = N \times \log_2(n)$	The absolute "information content" of the program, representing the mental storage required to process it.

The Predictive Metrics

Complexity, Effort, and Defects

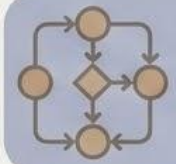
Forecasting Maintainability

Metric	Formula	What it Indicates
 Program Level (L)	$L = \frac{2}{n_1} \times \frac{n_2}{N_2}$	• The abstraction level of the code. An inverse measure of complexity (Lower L = Higher Complexity).
 Programming Effort (E)	$E = \frac{V}{L}$	• The total mental effort required to understand, write, or rewrite the module.
 Estimated Bugs (B)	$B = \frac{V}{3000}$	• A predictive estimate of the total number of delivered defects hiding within the code.

Combining Source Code Metrics

Token Density vs. Structural Complexity

The Holistic View

Metric	Formula	What it Indicates
 Halstead Metrics (Synthesized Lexical Complexity)	Concept: Density & Vocabulary ($f(N, n)$)	Measures "Lexical Complexity," representing the sheer density and vocabulary of the code. Higher scores indicate a larger mental dictionary.
 Cyclomatic Complexity ($V(G)$)	$V(G) = E - N + 2$	Measures "Structural Complexity," calculating the absolute number of linearly independent paths. Higher scores indicate greater logic branch complexity.

The Golden Rule for QA

High Halstead Effort + High Cyclomatic Complexity = Maximum Testing Priority.

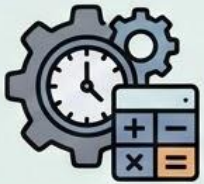
- Code in this category must be aggressively refactored.

Metrics for Testing and Maintenance

Guiding the QA Strategy

The Shift in Focus: We are moving from predicting design quality to actively predicting the testing effort required to guarantee that quality.

The Core Objectives

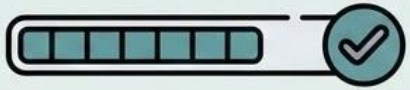


Predictive Effort

Use historical math to estimate how much time QA will need.

Targeted Testing

Use Object-Oriented metrics to show QA exactly where to look for bugs.



Coverage Assurance

Measure how thoroughly the testing was actually performed.

Predicting Effort and Defects

Applying Halstead and Cyclomatic Complexity

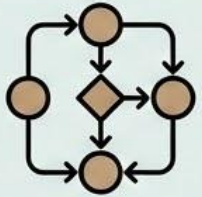
Mental Testing Effort

Halstead's Programming Effort (E) directly correlates with the mental effort required to test the software. Higher Volume (V) → Exponentially more test cases required.

Defect Prediction

Estimated Bugs (B) = $\frac{V}{3000}$.

(Example: A module with a Volume of 15,000 has approximately 5 predicted, hidden defects.) QA must find them before stopping.



Path Coverage

Cyclomatic Complexity ($V(G)$) provides the absolute minimum number of test cases required to achieve basic path coverage. Ensures every line of logic is executed at least once.

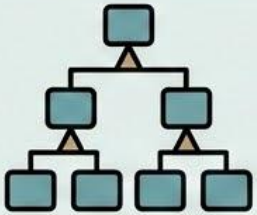
Guiding OO Testing

Using the CK Suite to Target QA Effort

We use the CK Metrics Suite to highlight exact risk areas for the QA team:

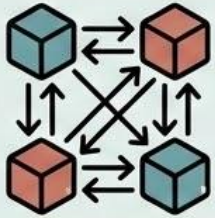
High WMC

The class is heavy. A massive number of unit test cases are required for this single class.



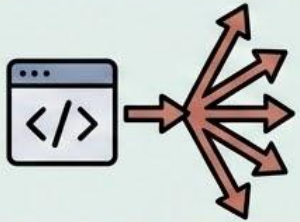
High DIT

Deep inheritance. QA must rigorously test how inherited behavior acts specifically within the subclass context.



High CBO

Tight coupling. Unit testing is insufficient; massive Integration Testing is required to cover all collaborations.



High RFC

Massive chain reactions. QA needs complex test cases to trace all possible responses a single message.

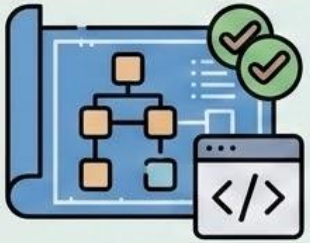
High LCOM

Low cohesion. Testing will be chaotic; strongly recommend splitting the class before QA begins.

Measuring QA Success

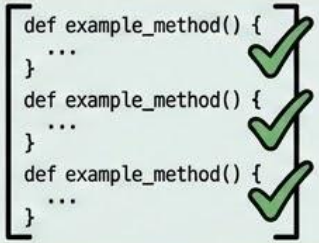
Test Coverage Metrics: “Are We Testing Enough?”

How do we mathematically prove that our QA process was thorough?



Class Coverage

The percentage of total classes in the system that have dedicated test suites.

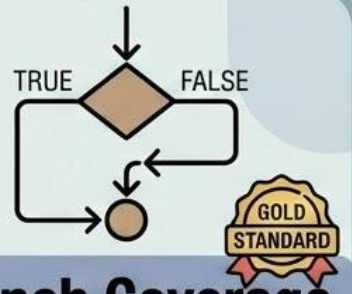


Method Coverage

The percentage of individual methods that have been executed by tests.

Statement Coverage

The percentage of absolute lines of code that were executed during the testing phase.



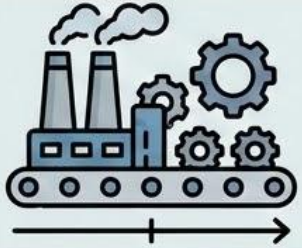
Branch Coverage

The percentage of decision branches (e.g., the true AND sides of an if statement) that were executed.

Metrics for Maintenance

Monitoring Long-Term Code Viability

Measuring and maintaining software viability over its long-term lifecycle.



Post-Release Focus

The majority of a software's lifecycle and total cost occurs after release during the maintenance phase.

Quantify Sustainability

Use specific code metrics to quantifiably monitor the ongoing maintainability of the codebase.

Strategic Forecasting

Accurately predict future effort and cost required to fix bugs and add new features based on current complexity.

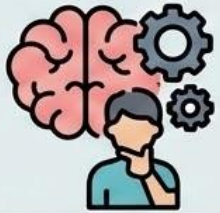
Refactoring Triggers

Identify exact thresholds where code must be refactored rather than patched to ensure long-term viability.

The Ultimate Composite Score

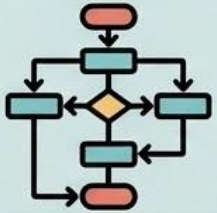
The Maintainability Index (MI) Combining Size, Complexity, and Volume

The Maintainability Index (MI) is a widely used composite metric that combines three core measurements we discussed earlier:



Halstead Volume (*V*)

The mental effort required to understand the code.



Cyclomatic Complexity (*V(G)*)

The structural complexity and decision paths.

Lines of Code (*LOC*)

The raw physical size of the module.

The Simplified Formula

$$MI = 171 - 5.2 \times \ln(V) - 0.23 \times V(G) - 16.2 \times \ln(LOC)$$

(Note: ln represents the natural logarithm)

Knowing When to Refactor

Highly Maintainable

Score: MI > 85

Proceed normally. The code is clean and easy to update.

Moderately Maintainable

Score: 65 ≤ MI ≤ 85

Proceed with caution. Technical debt is accumulating; monitor closely during the next update.

Difficult to Maintain

Score: MI < 65

Refactor required. The code is too brittle or complex; patching it will likely introduce new defects.

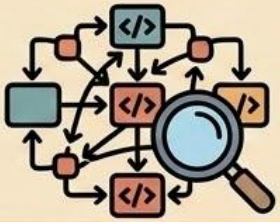
Maintenance Effort Metrics

Tracking Real-World Maintenance Costs

Beyond the static code, we must track the actual operational effort of the maintenance team:

Mean Time to Repair (MTTR)

The average clock time it takes a developer to successfully diagnose, fix, and verify a reported defect.



Change Density

The number of lines of code changed per Function Point.

(Indicates how intrusive updates are to the existing system).

Defect Arrival Rate

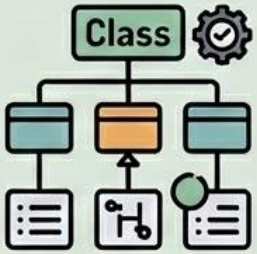
The raw number of new defects reported by users per time period (e.g., bugs per week) post-release.

(A rising rate indicates systemic failure or poor previous patches)

Conclusion & Key Takeaways

Quantifying the Design Phase

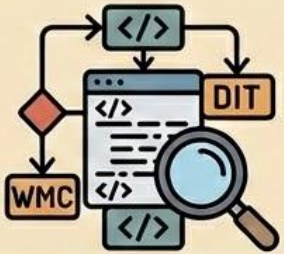
OO Design Metrics



OO Design Metrics

Provide quantitative insight into architectural and class-level quality.

CK Metrics



CK Metrics

Predict maintainability and testability through metrics like WMC, DIT, NOC, CBO, RFC, and LCOM.

MOOD Metrics

MOOD Metrics

Evaluate system-wide encapsulation (MHF, AHF), inheritance (MIF, AIF), and polymorphism (PF).

Lorenz & Kidd

Lorenz & Kidd

Establish practical, project-level rules for class size and inheritance.

Key Takeaways

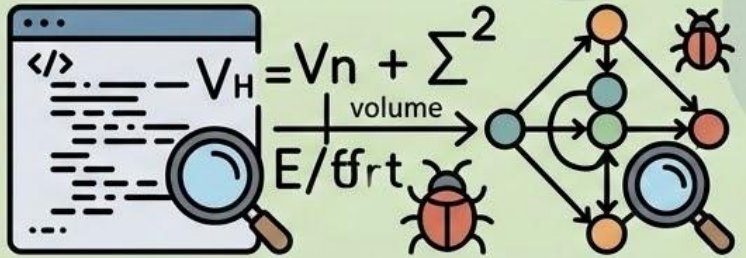
WebApps and Source Code

WebApp Design Metrics

Acknowledge the unique nature of the web.

Focus heavily on managing Content, optimizing Navigation, ensuring consistent Presentation, and scaling Performance.

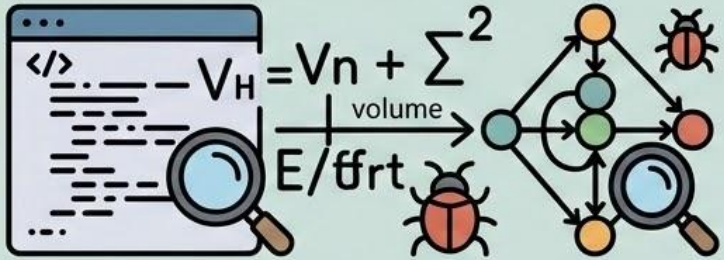
Source Code Metrics



- Halstead Metrics: Treat code as language to measure program volume, effort, and mathematically predict hidden bugs.
- Cyclomatic Complexity (V(G)): Measures the structural path complexity to define exact testing requirements.

Key Takeaways
Testing and Maintenance

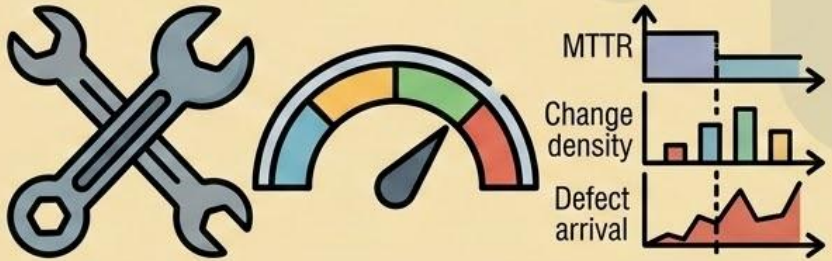
Testing Metrics



Utilize Halstead and Cyclomatic Complexity to predict minimum testing effort and test case volume.

Use the CK metrics suite to actively prioritize where QA should focus their Object-Oriented integration testing.

Maintenance Metrics



- **Maintainability Index (MI):** The ultimate composite score combining, volume, com and size to track long-term code health.
- **Operational Metrics:** Track MTTR (Mean Repair), change density, and the post-re to monitor the reality of team friction.

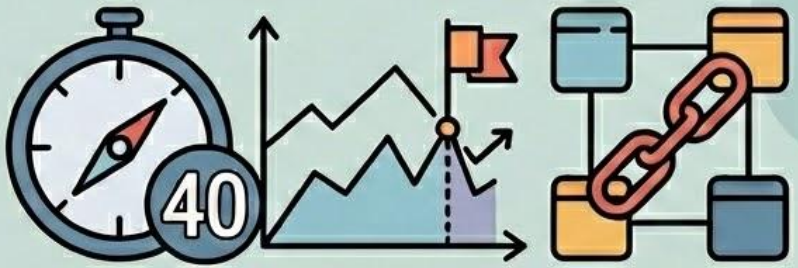
Slide 4: The Final Word

The Final Word: Steering Toward Quality

Acknowledge the Signal

- Metrics reveal *where* problems exist (Diagnostic).
- They serve as warning signals, not instant solutions.

Measure Wisely & Interpret



- Use metrics like low Maintainability Index (40) as explicit warnings.
- High Object Coupling signals potential trouble.
- MEASURE, GUIDE, AND CONTEXT: Use data to improve quality, not to judge.